

Guide de Déploiement Sécurisé - StatDirm

Ce guide vous accompagne étape par étape pour déployer StatDirm de manière sécurisée en production.

Prérequis

- Serveur Linux (Ubuntu/Debian recommandé)
- Docker et Docker Compose installés
- Domaine configuré avec DNS pointant vers le serveur
- Accès root ou sudo

Étape 1 : Configuration de l'Authentification

1.1 Générer le fichier .htpasswd

```
# Installer apache2-utils si nécessaire
sudo apt-get update
sudo apt-get install apache2-utils

# Générer le fichier .htpasswd
sudo htpasswd -c /etc/nginx/.htpasswd admin
# Entrer le mot de passe deux fois quand demandé

# Pour ajouter d'autres utilisateurs (sans -c pour ne pas écraser)
sudo htpasswd /etc/nginx/.htpasswd utilisateur2
```

1.2 Configurer Nginx

1. Copier le fichier `nginx.conf` dans le conteneur ou monter un volume
2. Décommenter les lignes d'authentification dans `nginx.conf` :

```
auth_basic "Accès restreint - DIRM Méditerranée";
auth_basic_user_file /etc/nginx/.htpasswd;
```

Étape 2 : Configuration HTTPS

2.1 Obtenir un certificat SSL avec Let's Encrypt

```
# Installer Certbot
sudo apt-get install certbot python3-certbot-nginx

# Obtenir le certificat (remplacer votre-domaine.fr)
sudo certbot --nginx -d votre-domaine.fr

# Le certificat sera automatiquement renouvelé
```

2.2 Configuration manuelle (alternative)

Si vous avez déjà un certificat :

1. Placer les fichiers dans `/etc/nginx/ssl/` :
 - `cert.pem` (certificat)
 - `key.pem` (clé privée)
2. Décommenter la section HTTPS dans `nginx.conf`
3. Mettre à jour les chemins des certificats

Étape 3 : Configuration Docker Sécurisée

3.1 Créer le fichier .env

```
cp .env.example .env
nano .env
```

Configurer :

```
VITE_APP_PASSWORD=votre-mot-de-passe-fort-et-unique
NODE_ENV=production
```

3.2 Construire et lancer les conteneurs

```
# Construire l'image
docker-compose -f docker-compose.prod.yml build

# Lancer en production
docker-compose -f docker-compose.prod.yml up -d
```

```
# Vérifier les logs
docker-compose -f docker-compose.prod.yml logs -f
```

Étape 4 : Configuration du Firewall

4.1 UFW (Ubuntu/Debian)

```
# Autoriser SSH (IMPORTANT : faire avant de bloquer tout)
sudo ufw allow 22/tcp

# Autoriser HTTP et HTTPS
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Activer le firewall
sudo ufw enable

# Vérifier le statut
sudo ufw status
```

4.2 FirewallD (CentOS/RHEL)

```
# Autoriser HTTP et HTTPS
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
```

Étape 5 : Monitoring et Logs

5.1 Vérifier les logs Nginx

```
# Logs d'accès
sudo tail -f /var/log/nginx/access.log

# Logs d'erreur
sudo tail -f /var/log/nginx/error.log
```

5.2 Logs Docker

```
# Logs de l'application
docker-compose -f docker-compose.prod.yml logs -f app
```

```
# Logs des 100 dernières lignes
docker-compose -f docker-compose.prod.yml logs --tail=100 app
```

✅ Étape 6 : Vérifications de Sécurité

6.1 Tester l'authentification

1. Accéder à l'application : `https://votre-domaine.fr`
2. Vérifier que la popup d'authentification apparaît
3. Tester avec un mauvais mot de passe (doit être refusé)
4. Tester avec le bon mot de passe (doit fonctionner)

6.2 Vérifier les headers de sécurité

```
curl -I https://votre-domaine.fr
```

Vérifier la présence de :

- `X-Frame-Options: DENY`
- `X-Content-Type-Options: nosniff`
- `Strict-Transport-Security: max-age=31536000`
- `Content-Security-Policy`

6.3 Tester le blocage des fichiers sensibles

```
# Doit retourner 403 Forbidden
curl https://votre-domaine.fr/src/data/agents.json
```

6.4 Tester le rate limiting

```
# Envoyer 30 requêtes rapidement
for i in {1..30}; do curl -I https://votre-domaine.fr; done

# Vérifier dans les logs qu'il y a des erreurs 429 (Too Many Requests)
```

Étape 7 : Maintenance et Mises à Jour

7.1 Mises à jour régulières

```
# Mettre à jour les packages système
sudo apt-get update && sudo apt-get upgrade

# Mettre à jour les dépendances npm
npm audit
npm audit fix

# Reconstruire les conteneurs Docker
docker-compose -f docker-compose.prod.yml build --no-cache
docker-compose -f docker-compose.prod.yml up -d
```

7.2 Sauvegardes

Créer un script de sauvegarde :

```
#!/bin/bash
# backup.sh

BACKUP_DIR="/backups/statdirm"
DATE=$(date +%Y%m%d_%H%M%S)

mkdir -p $BACKUP_DIR

# Sauvegarder les données
cp -r src/data $BACKUP_DIR/data_$DATE

# Sauvegarder la configuration
cp nginx.conf $BACKUP_DIR/nginx_$DATE.conf
cp docker-compose.prod.yml $BACKUP_DIR/docker-compose_$DATE.yml

# Compresser
tar -czf $BACKUP_DIR/backup_$DATE.tar.gz $BACKUP_DIR/*_$DATE*

# Supprimer les anciennes sauvegardes (garder 30 jours)
find $BACKUP_DIR -name "backup_*.tar.gz" -mtime +30 -delete

echo "Sauvegarde terminée : $BACKUP_DIR/backup_$DATE.tar.gz"
```

Ajouter au crontab pour exécution quotidienne :

```
crontab -e
# Ajouter : 0 2 * * * /chemin/vers/backup.sh
```

En Cas de Problème

Problème : Impossible de se connecter après configuration

1. Vérifier les logs : `docker-compose logs app`
2. Vérifier que `.htpasswd` est bien monté dans le conteneur
3. Vérifier les permissions du fichier `.htpasswd`

Problème : Certificat SSL non valide

1. Vérifier que le domaine pointe bien vers le serveur
2. Vérifier que les ports 80 et 443 sont ouverts
3. Relancer certbot : `sudo certbot renew`

Problème : Rate limiting trop strict

Ajuster dans `nginx.conf` :

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=20r/s;
```

Support

Pour toute question ou problème, consulter [SECURITE.md](#) ou contacter l'équipe technique.